



Report

Robot Mapping

Dinh-Son Vu

Table of Contents

1. Pre-requisites	3
2. Tasks	3
3. Explanations	4
4. Packages, nodes, and topics	5
5. Future works	6

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is simulation only. The hardware is not required.

2. Tasks

Lidar is the most important sensor for autonomous robots and is the main transition between a simple Arduino robot to a more sophisticated platform.

Tasks	Code
Install the slam toolbox	<code>sudo apt install ros-jazzy-slam-toolbox</code>
Clone the repo from github	<code>git clone https://github.com/ROS2-rmitbot/lesson4_ws.git</code>
Navigate to the workspace	<code>cd ~/lesson4_ws</code>
Remove the folders build install log	<code>rm -rf build install log</code>
Build the packages	<code>colcon build</code>
Source the workspace	<code>source install/setup.bash</code>
Open VSCode	<code>code .</code>
Launch the bringup launch	<code>ros2 launch rmitbot_bringup rmitbot.launch.py</code>

On Gazebo, it is possible to see the ray from the Lidar (Lidar Visualisation). Add some shapes in Gazebo and look at the cloud point on rviz. Then move the robot around: you should see the construction of the map.

However, the lidar ray cannot be rendered while using a VM in gazebo, however the /scan messages are still available in rviz and can be visualized and used to do the mapping.

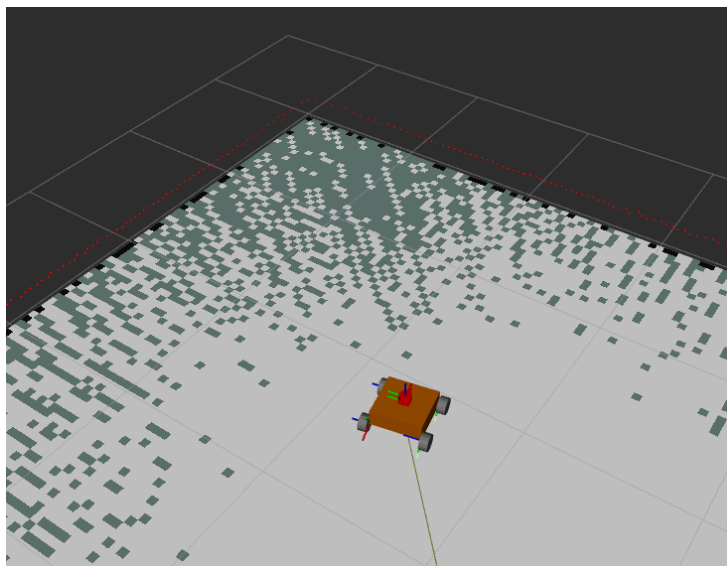


Figure 1: the robot detects the wall and maps its surroundings thanks to the Lidar and SLAM.

3. Explanations

A lidar has been added in the urdf, which includes a plugin in Gazebo. The lidar cannot construct a map if there is no obstacle, thus a simple world has been added which consists of four walls that construct an 8m x 8m environment. Slamtoolbox constructs the map of the surroundings and corrects the positioning of the robot as well, but it requires minimum movement from the robot: the principle is similar to a blind person walking in a room and touching the side of the wall to navigate in the environment. There are now three important frames:

- odom: initial pose of the robot from the robot_localization
- base_footprint: the current pose of the robot
- map: the inertial frame. The odometry from encoders only tends to drift: you want to travel 5m with your robot, but you may achieve only 4.95m. The imu can correct the heading, but not the linear accumulated error. The lidar, coupled with SLAM technique, allows the robot to know precisely its position relative to its environment. The map frame is the fixed frame, the odom frame is the initial pose of the robot that drifts due to accumulated error, and base_footprint is the actual pose of the robot.

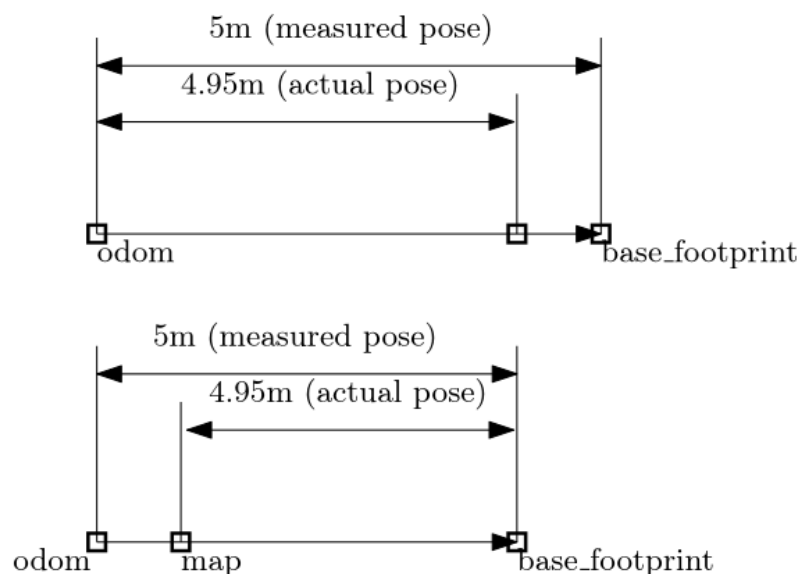


Figure 1: Comparison between odometry only and SLAM

- On the top, the robot position is based on odometry only. The measured pose of the robot given by odometry is 5m, but the real, actual pose is 4.95m: the robot has drifted by 0.05m.
- On the bottom, the odometry still gives a measurement of 5m, but the relative position between odom and map is updated by the lidar. The relative position between the map and the robot is thus correct.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu
rmitbot_mapping	slam	Launch the lidar and slam

Package	Topic	Description
ROS2 standard	/parameter_events /rosout	n/a n/a
rviz	/clicked_point /goal_pose /initialpose	Click point in rviz Target pose of the robot (for nav2) Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description /tf /tf_static	Description of the robot in xml format Dynamic tf (odom --> base_footprint) Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity /controller_manager/introspection_data/full /controller_manager/introspection_data/names /controller_manager/introspection_data/values	For debugging only
ros2_controllers	/joint_states /rmitbot_controller/cmd_vel /rmitbot_controller/odom /rmitbot_controller/cmd_vel_out /joint_state_broadcaster/transition_event /dynamic_joint_states /rmitbot_controller/transition_event	Important: pos, vel, effort of the joint Important: Cartesian vel command Important: odometry (pose of the robot)
robot_localization	/imu/out /odometry/filtered	Imu data (actually from gazebo, but used in robot_localization) Odom from ekf: filtered and less prone to drift than /rmitbot_controller/odom

slamtoolbox	<code>/map</code> <code>/map_metadata</code> <code>/map_updates</code> <code>/slam_toolbox/feedback</code> <code>/slam_toolbox/graph_visualization</code> <code>/slam_toolbox/scan_visualization</code> <code>/slam_toolbox/transition_event</code> <code>/slam_toolbox/update</code>	Loop closure: pose of the odom
--------------------	--	--------------------------------

5. Future works

- So far, the robot is spawned inside the unknown virtual environment in gazebo. The world is just an 8x8m room. It would be appreciated to build a more complicated world to test more reliable scenarios.